Execution-Grounded LLM Coding Agents: From Code Generation to Autonomous Software Engineering

Tong Zhu
Student ID 225040538

CSC6032 Advanced Operating System

School of Data Science
The Chinese University of Hong Kong, Shenzhen

April 2026

Abstract

Large language models have progressed from code completion systems into agents that can act inside a real software environment. This dissertation examines that transition under the topic "Execution-Grounded LLM Coding Agents: From Code Generation to Autonomous Software Engineering." The argument is developed through three milestone systems. Codex establishes code-specialized large language models as a serious direction for program synthesis and introduces HumanEval, which evaluates functional correctness by executing generated programs. CodeAct argues that executable code should not be treated merely as the final answer; instead, code can become the agent's native action language for interacting with tools and external state. OpenHands turns this principle into a general platform with sandboxed runtimes, shell and browser access, event streams, reusable skills, and benchmark-driven evaluation for software engineering tasks.

The central claim of this dissertation is that execution changes role across the three systems: it begins as an evaluation oracle, becomes part of the reasoning loop, and finally becomes the runtime substrate of an autonomous software engineering system. That progression explains why the topic is strongly relevant to operating systems and computer systems. Once agents depend on execution, their capabilities also depend on process management, filesystem state, sandboxing, reproducibility, observability, and permission control. The dissertation therefore uses the three works as evidence for a single thesis about execution-grounded software agents rather than as isolated model improvements. By analyzing the methodology, benchmark evidence, systems implications, and remaining limitations of each stage, the dissertation argues that modern coding agents should be understood as interactive software systems rather than only stronger code generators.

Contents

Abstract **i**

1 Introduction **1**

1.1 Research Background 1

1.2 Research Problem and Thesis Statement 1

1.3 Organization of the Dissertation 2

2 Technical Foundations and Research Context **3**

2.1 From Program Synthesis to Functional Evaluation 3

2.2 From Tool Use to Agent-Computer Interfaces 3

2.3 From Software Tools to Autonomous Software Engineering Platforms 4

2.4 Positioning of the Core Works 4

3 Codex: The Foundation of Execution-Based Code Generation **5**

3.1 Research Objective and Method 5

3.2 HumanEval and the Meaning of Execution 5

3.3 Key Quantitative Findings 6

3.4 Limitations of the Codex Stage 7

3.5 Contribution to the Dissertation Topic 7

4 CodeAct: Executable Code as the Agent Action Space **8**

4.1 Why Action Representation Matters 8

4.2 Code as Action and Interpreter-Centered Interaction 8

4.3 Evidence from M^3ToolEval and Related Benchmarks 9

4.4 Limits of the CodeAct Stage 9

4.5 Contribution to the Dissertation Topic 10

5 OpenHands: A Runtime Platform for Autonomous Software Engineering **11**

5.1 Platform Goal and Architectural Perspective 11

5.2 Runtime, Sandboxing, and Systems Abstractions 12

5.3 Evaluation on Software Engineering Tasks 12
5.4 Remaining Limitations 13
5.5 Contribution to the Dissertation Topic 13

6 Performance Evaluation and Comparative Analysis 14
6.1 How Benchmark Logic Evolves Across the Three Papers 14
6.2 Correlation of the Three Papers 14
6.3 The Dissertation Argument Revisited 15
6.4 Open Research Challenges 16
6.4.1 Long-Horizon Reliability 16
6.4.2 Sandboxing Versus Capability 16
6.4.3 Reproducibility and Evaluation Cost 16
6.4.4 Human Oversight and Collaboration 16

7 Conclusion 17

List of Figures

3.1 HumanEval performance reported in the Codex paper. The figure shows strong gains from code-specialized training and additional gains from repeated sampling and selection [1]. 6

4.1 Zero-shot CodeAct results comparing code, JSON, and text action formats. Executable code actions consistently improve success rate on multi-turn tasks while often reducing the number of required turns [4]. 9

5.1 OpenHands architecture. The paper separates agent logic, event-stream state management, and runtime execution inside a sandboxed environment [5]. . . 11

List of Tables

2.1 Positioning of the three core works within the dissertation topic. 4

3.1 Representative Codex results on HumanEval. 6

4.1 Representative CodeAct results on M^3ToolEval for `gpt-4-1106-preview`. . 9

5.1 Representative OpenHands results reported in the paper. 12

6.1 Evolution of benchmark design across the three papers. 14

6.2 How each paper contributes to the dissertation topic. 15

Chapter 1

Introduction

1.1 Research Background

Large language models (LLMs) have substantially changed the landscape of code intelligence. Early work in this area focused on whether a model could complete a function from a docstring or generate a small program fragment that looked plausible. That question remains important, but it is no longer sufficient. Modern coding agents are increasingly expected to inspect repositories, execute commands, interpret failures, edit files, and continue iterating until a software task is completed. The research problem therefore shifts from static code generation to stateful interaction with a computing environment.

This dissertation develops that transition through three papers that form a tightly connected technical narrative. *Evaluating Large Language Models Trained on Code* by Chen *et al.* [1] provides the foundation by showing that code-trained language models can solve non-trivial programming tasks and by proposing HumanEval as an execution-based benchmark. *Executable Code Actions Elicit Better LLM Agents* by Wang *et al.* [4] advances the field by treating executable code as the action representation of an agent. *OpenHands: An Open Platform for AI Software Developers as Generalist Agents* by Wang *et al.* [5] turns that interaction paradigm into a runtime platform for autonomous software engineering.

1.2 Research Problem and Thesis Statement

The guiding problem of this dissertation is straightforward: *how did the field move from code generation models to execution-grounded software engineering agents, and why does that movement create a meaningful bridge to computer systems research?*

The thesis statement of this dissertation is that execution is the key technical thread connecting all three papers. In Codex, execution is primarily an oracle for measuring correctness. In CodeAct, execution becomes part of the agent's internal loop because the model writes and runs code to gather feedback. In OpenHands, execution is elevated further into a managed runtime that coordinates shell commands, notebook cells, browser actions, filesystems, and sandbox policies. This progression shows that coding-agent capability is not only a property of the model; it is also a property of the execution environment and the interface exposed by that environment. From a systems perspective, this shift turns process creation,

workspace mounting, permission control, side-effect isolation, runtime state management, and action logging into part of the research problem rather than mere implementation details.

1.3 Organization of the Dissertation

The remainder of this dissertation is organized as follows. Chapter 2 establishes the broader research context around code generation, agent-computer interfaces, and autonomous software engineering. Chapters 3, 4, and 5 analyze Codex, CodeAct, and OpenHands as three sequential stages in the dissertation argument. Chapter 6 synthesizes their benchmark logic, extracts their shared technical trajectory, and explains how each stage contributes to the thesis. Chapter 7 concludes the dissertation and summarizes open challenges.

Chapter 2

Technical Foundations and Research Context

2.1 From Program Synthesis to Functional Evaluation

The modern literature on coding agents begins with neural code generation, but the decisive methodological change is not only that models became larger. The more important change is that correctness started to be evaluated by execution instead of by text similarity. Codex is the clearest early example of this shift. Chen *et al.* argue that code quality should be measured by whether a generated function passes tests, not by whether it resembles a reference program [1]. This view is now standard in the field because executable correctness is much more informative than lexical overlap for programming tasks.

Execution-based evaluation matters because code is a dual artifact: it is written as text, but its actual value is operational. A program is useful when it compiles, runs, and produces the intended behavior in a concrete environment. Once evaluation is grounded in execution, the research direction naturally begins to ask whether execution can also be used during reasoning, not only after generation.

2.2 From Tool Use to Agent-Computer Interfaces

CodeAct marks a second conceptual step. Rather than forcing an agent to communicate through rigid text or JSON tool calls, Wang *et al.* propose executable code as a more expressive action language [4]. This idea reframes the agent-computer interface as an algorithmic choice. A model that can emit Python programs can compose control flow, intermediate variables, library imports, and external calls within one action. As a result, interaction with the environment becomes less fragmented and more similar to the way human developers use a computational workspace.

This shift is relevant to systems research because action representation determines what the runtime must support. Once code becomes the action language, interpreter state, error handling, resource access, and side-effect control become part of the agent design itself. The interface is no longer a neutral wrapper around a model; it is part of the mechanism that generates capability.

2.3 From Software Tools to Autonomous Software Engineering Platforms

A third research thread asks how agentic capabilities should be packaged into platforms that can support realistic development tasks. SWE-Bench formalizes repository-level issue resolution as a benchmark, while SWE-Agent shows that agent-computer interfaces strongly influence software engineering performance [2, 6]. OpenHands extends this line of work by proposing a general platform for AI software developers with a unified runtime, event streams, built-in actions, reusable skills, and evaluation harnesses [5]. The environment itself becomes a first-class design object.

This platform perspective is what makes the literature strongly relevant to operating systems. The runtime must coordinate a shell, an interpreter, a browser, mounted workspaces, and isolation boundaries. Reproducibility, resource control, observability, and safety become inseparable from model performance.

2.4 Positioning of the Core Works

The three core works form a coherent progression rather than a loose collection of references. Codex is the foundational paper because it proves that code-specialized LLMs and execution-based benchmarks matter. CodeAct is the systems bridge because it treats executable code as the native action interface of an agent. OpenHands is the recent systems platform because it operationalizes that idea in a reusable runtime for autonomous software engineering. Together, they support the dissertation topic at three levels: capability foundation, interaction design, and runtime realization.

Table 2.1: Positioning of the three core works within the dissertation topic.

Paper	Primary Contribution	Role in the Topic	Systems Connection
Codex	Demonstrates large-scale code generation and introduces HumanEval.	Foundation for execution-grounded code intelligence.	Safe execution harness is required to validate generated programs.
CodeAct	Replaces fixed tool schemas with executable code actions.	Systems bridge from generation to interactive agency.	Interpreter state, feedback, and action interfaces become central.
OpenHands	Builds a general runtime platform for AI software developers.	Recent realization of autonomous software engineering.	Sandboxing, mounted workspaces, shell access, browser access, and event tracing are first-class.

Chapter 3

Codex: The Foundation of Execution-Based Code Generation

3.1 Research Objective and Method

Codex addresses the first question in the narrative: can a large language model trained on source code generate functionally correct programs from natural language specifications? Chen *et al.* fine-tune GPT-style models on publicly available code from GitHub and evaluate the resulting models on Python program synthesis tasks [1]. The central contribution is not only a stronger model, but also a clearer experimental framework for judging code generation.

The paper focuses on function synthesis from docstrings. This framing is narrower than later repository-level software engineering tasks, but it is appropriate for a foundational study. It isolates the model's ability to translate a concise natural-language specification into executable code while avoiding the additional complexity of long interaction histories or external tools.

3.2 HumanEval and the Meaning of Execution

The methodological breakthrough of Codex lies in HumanEval. Rather than scoring programs with BLEU or another text-overlap metric, the paper evaluates correctness by executing generated functions against hidden tests [1]. This decision is conceptually important because it treats code as an operational artifact. It also creates a lasting benchmark that later work continues to use, either directly or as a reference point.

HumanEval contains 164 handwritten Python problems with function signatures, docstrings, reference solutions, and unit tests [1]. The benchmark is intentionally small but carefully curated. Its value lies in the fact that successful performance cannot be reduced to superficial lexical similarity. A candidate solution must actually run and satisfy the behavioral constraints encoded by the tests.

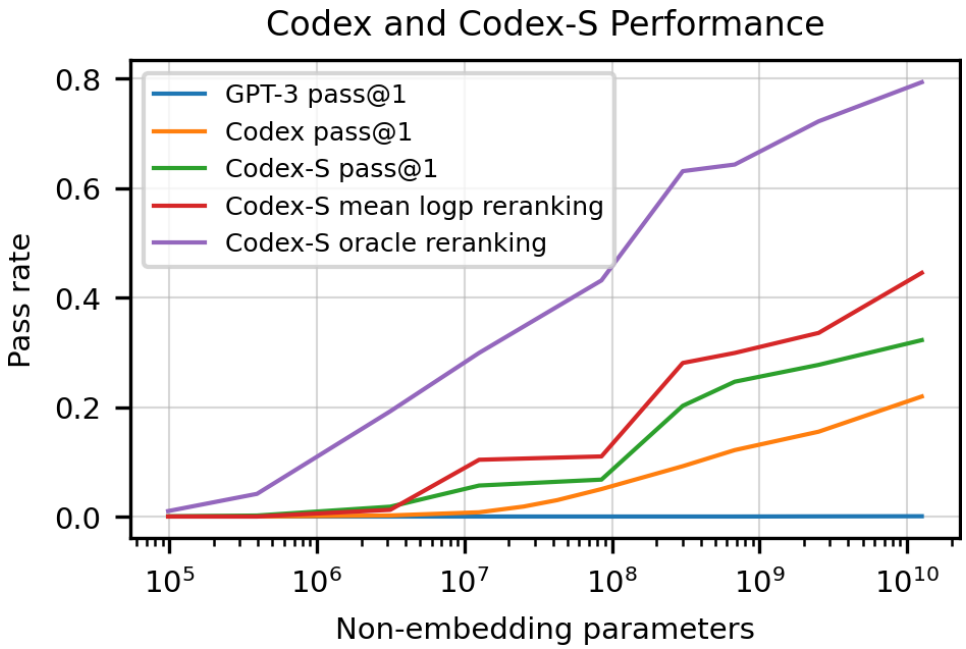

Figure 3.1: HumanEval performance reported in the Codex paper. The figure shows strong gains from code-specialized training and additional gains from repeated sampling and selection [1].

3.3 Key Quantitative Findings

Codex provides clear empirical evidence that code-specialized LLMs outperform general language models on program synthesis. The 12B Codex model solves 28.8% of HumanEval problems with one sample, while GPT-3 solves essentially none of the tasks in the same setup [1]. When the authors allow repeated sampling, performance rises further to 70.2% with 100 samples per problem. The fine-tuned Codex-S variant reaches 37.7% for one-sample evaluation, indicating that additional task-specific refinement continues to matter [1].

These results establish two lasting insights. First, code pretraining clearly transfers into functional programming competence. Second, one-shot accuracy understates model capability because search and candidate selection can recover correct solutions already present in the model's distribution. Later agent papers inherit both lessons: execution is the right oracle, and a model should be allowed to interact with that oracle more directly.

Table 3.1: Representative Codex results on HumanEval.

Setting	Result
Codex-12B, one sample	28.8% solved
Codex-S, one sample	37.7% solved
Codex-12B, 100 samples	70.2% solved
Evaluation principle	Functional correctness under hidden tests

3.4 Limitations of the Codex Stage

Codex is foundational, but it is still a pre-agent paper. The model receives a prompt and returns a completion; it does not maintain persistent runtime state, call tools during reasoning, or navigate a repository. Execution appears in the paper mainly as an evaluator rather than as part of the action loop. This means the paper answers the question "can LLMs generate useful code?" without yet answering the more difficult question "can LLMs behave like software workers inside a computing environment?"

Another limitation is that repeated sampling is used as an external search strategy rather than as an explicit interaction process. The model does not observe why a sample failed, read stack traces, or iteratively debug. That gap becomes the starting point for CodeAct.

3.5 Contribution to the Dissertation Topic

Within the dissertation topic, Codex contributes the foundational claim that execution-based correctness is the right lens for studying code intelligence. Without that premise, later execution-grounded agents would have a weaker justification. Codex therefore supplies the first stage of the dissertation argument: before execution can become the action language or the runtime substrate, it must first be established as the correct evaluation principle.

Chapter 4

CodeAct: Executable Code as the Agent Action Space

4.1 Why Action Representation Matters

CodeAct studies a different problem from Codex. The question is no longer whether a model can emit a correct final program, but whether the model should use executable code to interact with its environment during problem solving. Wang *et al.* argue that code has structural advantages over plain text or JSON-style tool calls because it naturally supports variables, control flow, library use, and composition [4].

This is a major conceptual shift. In Codex, code is the answer. In CodeAct, code is also the action. The model can write a Python snippet, execute it, inspect the result, and continue. Once this loop is available, external execution stops being a passive evaluator and becomes an active source of feedback for reasoning.

4.2 Code as Action and Interpreter-Centered Interaction

The core intuition behind CodeAct is that large language models are already strongly familiar with code because of pretraining. If the environment accepts executable code as an action, the agent can express a multi-step plan within one coherent program rather than across multiple rigid tool invocations [4]. The interpreter provides a shared substrate for state and observation: imported libraries, intermediate variables, and runtime errors all become accessible to the model through execution feedback.

From a systems perspective, this is the clearest bridge work in the dissertation. The success of the method depends on the shape of the agent-computer interface. A more expressive interface reduces action fragmentation and allows the environment to carry persistent state across turns. The paper therefore suggests that capability is a joint product of model intelligence and runtime design.

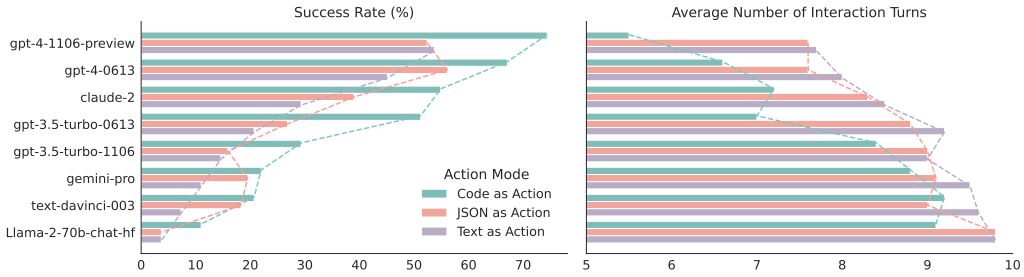

Figure 4.1: Zero-shot CodeAct results comparing code, JSON, and text action formats. Executable code actions consistently improve success rate on multi-turn tasks while often reducing the number of required turns [4].

4.3 Evidence from M^{3}ToolEval and Related Benchmarks

The most persuasive empirical evidence in CodeAct comes from the multi-turn benchmark M^{3}ToolEval. On this benchmark, the authors show that executable code actions outperform both JSON actions and text actions across a wide range of models [4]. For `gpt-4-1106-preview`, the paper reports a 74.4% success rate with CodeAct, compared with 52.4% for JSON actions and 53.7% for text actions [4]. The same setting also uses fewer interaction turns on average, which means the gain is not only in final accuracy but also in action efficiency.

The paper further shows that the idea can improve smaller open-source models through instruction tuning on agent-environment trajectories. This matters because CodeAct is not only a prompting trick for frontier closed-source systems. It also proposes a data and interface design that can be used to train more capable open agents.

Table 4.1: Representative CodeAct results on M^{3}ToolEval for `gpt-4-1106-preview`.

Action Format	Success Rate / Observation
Executable code	74.4% success
JSON actions	52.4% success
Text actions	53.7% success
Additional takeaway	Code actions also reduce average interaction turns

4.4 Limits of the CodeAct Stage

CodeAct proves the value of executable actions, but it does not yet provide a full software engineering platform. The runtime studied in the paper is centered on interpreter interaction rather than on a complete developer workstation. Realistic software tasks often require shell access, repository navigation, package management, browser use, and long-lived workspace state. Those concerns are acknowledged by the paper, but they are not yet the main implementation focus.

A second limit is scope. The benchmarks are convincingly multi-turn, yet they remain more controlled than repository-scale bug fixing or issue resolution. CodeAct therefore es-

tablishes the right interface principle while leaving open the question of how that principle should be engineered into a broader runtime. OpenHands takes that next step.

4.5 Contribution to the Dissertation Topic

Within the dissertation topic, CodeAct provides the essential systems bridge. It explains why execution should move from evaluation to action, and it demonstrates that the design of the action interface materially affects agent performance. This paper contributes the dissertation's second stage: execution is not merely how we score programs after generation; it is how agents should interact with the world while solving tasks.

Chapter 5

OpenHands: A Runtime Platform for Autonomous Software Engineering

5.1 Platform Goal and Architectural Perspective

OpenHands shifts the literature from interaction design to platform design. Wang *et al.* present it as an open platform for AI software developers that supports multiple agents, multiple runtimes, reusable skills, human interaction, and benchmark-driven evaluation [5]. The paper is important because it asks a systems question directly: if execution-grounded agents are going to solve realistic software tasks, what runtime abstractions do they need?

The answer is a modular architecture built around three concepts: agents, event streams, and runtimes. Agents produce actions. The event stream records the history of actions and observations. The runtime executes actions inside an isolated environment and returns the resulting observations. This design turns the interaction loop into a reusable systems substrate instead of a one-off experimental harness.

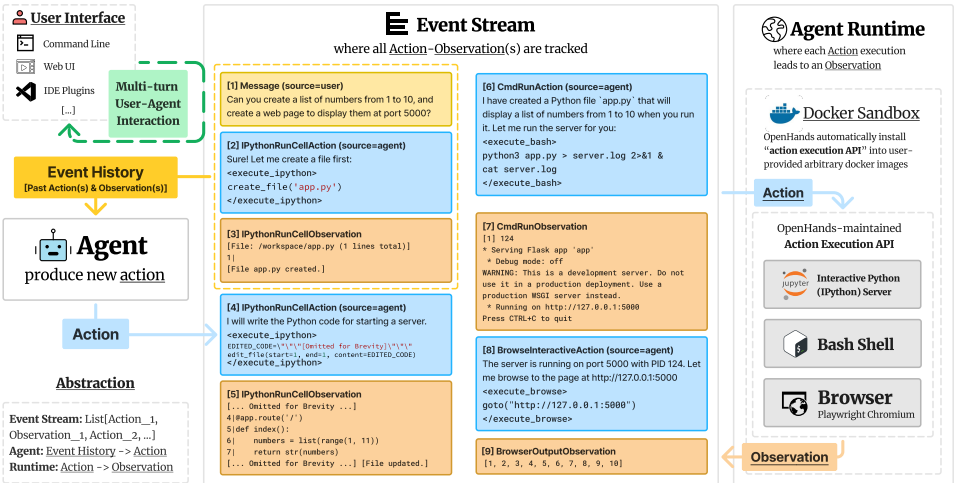

Figure 5.1: OpenHands architecture. The paper separates agent logic, event-stream state management, and runtime execution inside a sandboxed environment [5].

5.2 Runtime, Sandboxing, and Systems Abstractions

OpenHands generalizes the CodeAct idea beyond a single interpreter. The runtime can execute bash commands, run IPython notebook cells, and interact with a browser inside a sandboxed environment [5]. A configurable workspace is mounted into the container so that the agent can inspect and edit task-relevant files. This is precisely the kind of systems design that makes the platform relevant to operating systems.

Several design decisions stand out. First, the Docker sandbox is treated as a core primitive rather than an implementation footnote. This enables isolation, environment control, and a safer path for running arbitrary code. Second, the event stream stores the chronology of actions and observations, which supports reproducibility, debugging, and human inspection. Third, the platform includes reusable skills and an AgentHub, showing that the authors view agent capability as a combination of model behavior and runtime affordances.

These decisions are significant because they make environmental state explicit. Filesystem contents, shell outputs, browser state, and command history are no longer hidden behind the prompt. They become part of the system state that the agent can inspect and manipulate.

5.3 Evaluation on Software Engineering Tasks

OpenHands is evaluated on multiple benchmarks, but its most important evidence for this dissertation comes from software engineering tasks. On SWE-Bench Lite, the paper reports a 26.0% resolve rate for CodeActAgent v1.8 with `claude-3-5-sonnet` [5]. This result is competitive with specialized software engineering agents while being delivered by a more general platform. On the Python split of HumanEvalFix, the paper reports 79.3% bug-fixing performance for one CodeActAgent configuration [5, 3].

These benchmarks are more operational than HumanEval. They require repository understanding, file editing, command execution, and iterative validation against tests. The performance numbers are therefore not just about code synthesis quality. They reflect how well the model and runtime cooperate over longer-horizon tasks.

Table 5.1: Representative OpenHands results reported in the paper.

Benchmark	Reported Result
SWE-Bench Lite	26.0% resolve rate with `claude-3-5-sonnet`
HumanEvalFix (Python)	79.3% bug-fixing performance
Core runtime actions	Bash, IPython, browser, reusable skills
Systems implication	Capability depends on sandbox quality and workspace management

5.4 Remaining Limitations

OpenHands does not solve autonomous software engineering completely. Benchmark performance remains well below human expert performance, especially on long and ambiguous tasks. The platform must constantly balance capability against safety: broader shell and browser access makes the agent more useful, but it also increases risk. In addition, evaluation results depend on the quality of the runtime image, installed dependencies, benchmark harnesses, and task setup. In that sense, OpenHands exposes the practical reality that coding agents are as much systems artifacts as model artifacts.

5.5 Contribution to the Dissertation Topic

Within the dissertation topic, OpenHands provides the recent and mature realization of execution-grounded coding agents. It contributes the dissertation's third stage: execution is no longer just a correctness test or an action format, but the managed runtime substrate of an autonomous software engineering system. This is the point where the topic most clearly enters operating systems territory.

Chapter 6

Performance Evaluation and Comparative Analysis

6.1 How Benchmark Logic Evolves Across the Three Papers

The most visible progression across the three papers is the unit of evaluation itself. Codex uses HumanEval to measure whether a model can generate a correct function from a docstring. CodeAct uses multi-turn success rate to test whether executable actions help an agent solve interactive tasks. OpenHands adopts repository-scale and bug-fixing benchmarks that evaluate complete development workflows. As the benchmark becomes more realistic, the runtime must also become richer.

Table 6.1: Evolution of benchmark design across the three papers.

Paper	Representative Benchmark	What the Benchmark Measures	Environment Assumption
Codex	HumanEval	Functional correctness of synthesized functions	Safe harness for running hidden tests
CodeAct	API-Bank and M^3ToolEval	Multi-turn task completion under different action formats	Stateful interpreter that returns execution feedback
OpenHands	SWE-Bench Lite and HumanEvalFix	End-to-end repository repair and iterative bug fixing	Sandboxed workspace runtime with shell, notebook, and browser access

6.2 Correlation of the Three Papers

The three papers are strongly correlated at the level of both methodology and research ambition. Codex establishes that execution is the right correctness oracle. CodeAct notices that if execution is already the right oracle, then executable code should also be an efficient action

space. OpenHands then recognizes that a code-based action space needs a durable runtime architecture if it is to support realistic software engineering. In other words, the papers form a chain of increasingly explicit environmental grounding.

This correlation also explains why the three works can support a single dissertation argument. They are not simply adjacent in time. Each one answers a gap left by the previous stage. Codex leaves open the role of interactive feedback. CodeAct fills that gap but leaves open the question of a complete systems substrate. OpenHands addresses that remaining gap by building the runtime platform itself.

Table 6.2: How each paper contributes to the dissertation topic.

Paper	Contribution to the Topic	Bridge to Systems	Main Remaining Gap
Codex	Shows that code-specialized LLMs can generate functionally correct programs.	Introduces execution harnesses and test-based evaluation.	No persistent environment interaction or iterative debugging.
CodeAct	Shows that executable code is an effective action representation for agents.	Makes interpreter state and runtime feedback part of the reasoning loop.	Runtime remains narrower than a full software engineering workspace.
OpenHands	Packages execution-grounded agency into a reusable platform.	Makes sandboxing, workspaces, event streams, and shell access first-class.	Long-horizon reliability, cost, and safety remain unresolved.

6.3 The Dissertation Argument Revisited

The dissertation topic is "Execution-Grounded LLM Coding Agents: From Code Generation to Autonomous Software Engineering." The three papers support that topic in a precise sequential order.

First, Codex justifies the phrase "code generation" because it proves that LLMs trained on code can solve meaningful synthesis tasks and that success should be judged functionally. Second, CodeAct justifies the phrase "execution-grounded" because it demonstrates that executable code is not merely the output but also the mechanism by which the agent interacts with tools and receives feedback. Third, OpenHands justifies the phrase "autonomous software engineering" because it provides a runtime platform in which agents can act on repositories, execute commands, browse, and maintain structured interaction histories.

Accordingly, the dissertation advances a three-stage claim:

1. Large language models become useful for programming when correctness is grounded in execution rather than text similarity.

2. Once execution is accepted as the correctness oracle, executable code becomes a natural

and efficient agent action space.

3. Once code is used as an action space, autonomous software engineering depends on the quality of the surrounding runtime, including isolation, state management, observability, and tool access.

6.4 Open Research Challenges

Although the trajectory is promising, the comparative analysis also reveals several unresolved issues.

6.4.1 Long-Horizon Reliability

Even the strongest systems analyzed in this dissertation remain brittle on long tasks. Codex needs repeated sampling to expose hidden capability. CodeAct improves multi-turn behavior but does not eliminate planning errors. OpenHands still resolves only a minority of SWE-Bench Lite tasks. Progress therefore depends not only on model quality, but also on better state management, recovery strategies, and task decomposition.

6.4.2 Sandboxing Versus Capability

Execution is valuable precisely because it enables real action, yet real action also creates operational risk. A powerful runtime exposes shells, files, and networks; a safe runtime constrains them. The balance between capability and containment is likely to remain central for future coding-agent systems.

6.4.3 Reproducibility and Evaluation Cost

As benchmarks move from isolated functions to repositories, environmental reproducibility becomes harder. Installed dependencies, runtime images, task harnesses, and workspace state all influence results. OpenHands also makes explicit that evaluation has a cost in tokens, containers, browser sessions, and test executions [5]. This turns agent evaluation into an operational systems problem as much as a modeling problem.

6.4.4 Human Oversight and Collaboration

A final challenge is how autonomy should be combined with human supervision. Practical software engineering often benefits from interruption, inspection, and clarification rather than full independence. The next generation of agent platforms will need stronger support for collaborative workflows in which humans and agents share control of the runtime.

Chapter 7

Conclusion

This dissertation has argued that execution is the organizing principle behind the transition from code generation to autonomous software engineering. Codex establishes the foundation by proving that code-trained LLMs can solve functional programming tasks and by making execution-based evaluation central. CodeAct moves the field from generation to interaction by treating executable code as the agent's native action language. OpenHands then scales that interaction paradigm into a general runtime platform for autonomous software engineering.

The most important conclusion is that execution changes role across the field. It begins as an oracle for correctness, becomes part of the reasoning loop, and finally becomes the runtime abstraction that organizes the whole workflow. This progression explains why the topic naturally bridges machine learning, software engineering, and operating systems. Once coding agents act inside real environments, their capabilities depend on sandboxing, filesystems, process execution, action logging, and reproducible runtime design.

The dissertation therefore supports a concrete thesis: progress in LLM coding agents is increasingly determined by the quality of the execution environment, not only by the strength of the underlying model. The three papers are tightly related, they represent a clear chronological and conceptual progression, and they offer a concrete systems bridge from LLM coding agents to the concerns of advanced operating systems. Future work in this area will likely depend not only on stronger foundation models, but also on better runtime architectures, safer execution policies, and more realistic evaluation environments.

Bibliography

[1] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating Large Language Models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[2] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-Bench: Can Language Models Resolve Real-World GitHub Issues? In *The Twelfth International Conference on Learning Representations*, 2024.

[3] Niklas Muennighoff, Qian Liu, Armel Zebaze, Qinkai Zheng, Binyuan Hui, Terry Yue Zhuo, Swayam Singh, Xiangru Tang, Leandro von Werra, and Shayne Longpre. OctoPack: Instruction tuning code Large Language Models. *arXiv preprint arXiv:2308.07124*, 2023.

[4] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.

[5] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations*, 2025.

[6] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *arXiv preprint arXiv:2405.15793*, 2024.